1 Introduction

This document is intended as a small semantic benchmark for PDF/UA generation and structure recovery. It contains headings, paragraphs, lists, links, code blocks, a block quote, a table, a figure, and footnotes.

The main goal is to test whether common authoring pipelines preserve document structure from source to PDF and back again. For a general reference on Markdown, see the Markdown Guide.

1.1 Headings and Paragraphs

A structured document should preserve heading hierarchy, paragraph flow, and inline semantics. This paragraph contains **bold text**, *italic text*, and `#raw("inline code", lang: "text")` to test whether emphasis and code are preserved during conversion.

1.1.1 Inline Semantics

Semantic preservation matters because a heading is not just larger text, and a list is not just visually indented text. A machine-readable document should retain these distinctions so that downstream extraction tools can recover them reliably[1].

2 Lists

Below is an unordered list with nesting:

- First bullet item
- Second bullet item
  - Nested bullet item
  - Another nested bullet with a link to the W3C
- Third bullet item

Below is an ordered list with nesting:

1. Prepare the source file.
2. Export the document to PDF.
   1. Enable accessibility tags if the tool supports them.
   2. Record the export settings.
3. Validate the resulting PDF.
4. Compare the extracted output against the source.

3 Block quote

> A document can look visually correct while still being structurally broken. Accessibility and machine readability depend on the logical structure being preserved, not only on the page layout.

4 Code Block

The following shell command illustrates a simple conversion workflow:

```
pandoc doc-a-semantic-article.md -o doc-a.tex
lualatex doc-a.tex
```

A second code example in Python:

```
Python
from pathlib import Path
```

[1] In this benchmark, “structure” includes headings, lists, tables, figures, links, reading order, and metadata.

```
source = Path("doc-a-semantic-article.md")
print(f"Processing {source.name}")
```

5 Table Example

Element	Example	Purpose
Heading	‘# Introduction’	Encodes section hierarchy
Emphasis	‘*’	Marks strong importance
Link	‘’	Preserves navigation and references
Footnote	‘^1’	Adds supporting notes
Code	‘inline’	Distinguishes literal code from text

Table 1: Comparison of Markdown elements and their semantic purposes

6 Figure Example



Figure 1: Example figure used to test image handling, alt text, and caption preservation

7 Footnotes and Closing

This sentence includes a footnote for testing purposes.[2] This second sentence includes another footnote to test multiple notes in the same document.[3]

In summary, this document is meant to be simple, readable, and rich enough to test whether structure survives across authoring, PDF generation, validation, and extraction workflows.

[2] A footnote should remain associated with the correct reference point in the main text.

[3] Some pipelines preserve footnote text but lose the reference structure or numbering semantics.